Université Grenoble Alpes
L3 Informatique

Année 2026-2027

Architectures logicielles et matérielles
Semaine 3 TD (1)

Dans ce TD, nous préparons la première séance de TP sur carte STM32F4, qui sera consacrée à la prise en mains de l'environnement et à quelques opérations de base sur les ports GPIO.

Questions à traiter en début de séance. (remise à 8h15)

On rappelle que les **ports GPIO** sont des ports d'entrées/sorties à usage général permettant à un microcontrôleur de communiquer avec des périphériques externes. Certains d'entre eux sont directement connectés aux boutons poussoir, ou aux leds.
Il s'agit d'une **banque de ports d'E/S**. Sur notre carte, chaque port est un ensemble de **16 pins**, qui peuvent jouer indifféremment le rôle d'entrée ou de sortie. Chaque pin peut être configuré, entre autres, selon :

- une vitesse,
- un *mode* : mode *entrée (in)*, mode *sortie (out)*, ou deux autres modes dont nous ne parlerons pas ici. Cette configuration se fait **via la valeur de 2 bits dans un registre MODER** ("mode register").

Question 1.
Les ports GPIO étant des ports d'E/S à usage général, la configuration pour une utilisation donnée est essentielle.
- Rappeler la forme du registre MODER des ports GPIO (faire le schéma).
- Pourquoi y a-t-il 32 bits alors qu'un port est un ensemble de 16 pins ? expliquer soigneusement le rôle de ces 32 bits.

Question 2.
Nous n'utiliserons pas le même registre selon qu'on veuille faire une utilisation en entrée ou en sortie.
- Lorsqu'un port GPIO est utilisé en entrée, comment (où ?) récupère-t-on l'état de l'entrée ?
- Lorsqu'un port GPIO est utilisé en sortie, comment (où ?) modifiera-t-on l'état de cette sortie ?
- Rappeler la forme des registres IDR et ODR des ports GPIO (faire le schéma) en expliquant le rôle de leurs (16 bits de poids fort et) 16 bits de poids faible.

Question 3.
Ci-dessous les noms et valeurs de quelques macros C qui seront utiles dans cette séance et les suivantes :

```
GPIOA      0x40020000    adresse de base du port A
GPIOB      0x40020400    adresse de base du port B
GPIOC      0x40020800    adresse de base du port C
GPIOD      0x40020C00    adresse de base du port D

GPIOD_MODER    (* (volatile uint32_t *)0x40020C00 )
GPIOD_IDR      (* (volatile uint32_t *)0x40020C10 )
GPIOD_ODR      (* (volatile uint32_t *)0x40020C14 )
GPIOD_BSRR     (* (volatile uint32_t *)0x40020C18 )
```

- Quelle est l'adresse de base pour les registres du port GPIOD ?
- Comment se situent ses 4 registres par rapport à cette adresse de base ? (faire un schéma)
Note. Nous verrons l'utilité du registre BSRR plus loin.

Exercice 1.

Dans ce TD/TP, nous mettrons en place de quoi allumer/éteindre 4 **leds** colorées, qui sont connectées à 4 pins consécutifs d'un port GPIO, dans l'ordre suivant (le bit de poids fort est à

gauche) :

BLEU ROUGE ORANGE VERT

Un bit à 1 signifie que la led sera allumée, et éteinte si le bit est à 0.

Q1. Combien y a-t-il de possibilités d'allumage ?

Q2. Donner ces différentes valeurs en précisant les leds allumées dans chaque cas.

Exercice 2.

Plus précisément, les 4 **leds** sont connectées au port **GPIOD**. La led verte est connectée au pin 12 de ce port, la led orange est connectée au pin 13, la led rouge est connectée au pin 14, et la led bleue est connectée au pin 15.

Q1. Quelle est l'adresse du registre dans lequel sera faite la configuration ?
Quels sont les numéros des bits de ce registre qui seront utilisés pour cette configuration ?

Q2. Quelle est l'adresse du registre qui sera utilisé pour modifier l'allumage des leds ?
Quels sont les numéros des bits de ce registre qui seront utilisés pour choisir cet allumage ?

Q3. Afin de concrétiser les rôles de ces registres, nous allons ici examiner une trace d'exécution, réalisée sur carte STM32F4. Dans cette exécution, des contenus de registres sont observés ou modifiés.

Note importante. Pour simplifier l'observation et la compréhension, les affectations sont faites ci-dessous via gdb (le programme exécuté au lancement ne fait que procéder aux configurations nécessaires, jusqu'à l'appel à une fonction **`fonction1`**). Mais par la suite (exercice 3 et TP), bien entendu c'est dans le programme C lui-même que nous réaliserons les actions faites ici (allumage leds) !

Analyser la trace d'exécution ci-dessous et ses effets : répondre aux questions.

```
$ make debug        ← chargement sur la carte et exécution dans gdb (voir TP...)
echo "\n\nBoot from ROM : check P2 pin 21 Boot0 has NO JUMPER\n\n";sleep 2;
\n\nBoot from ROM : check P2 pin 21 Boot0 has NO JUMPER\n\n
...
target halted due to debug-request, current mode: Thread
Loading section .text, size 0xf30 lma 0x8000000
Loading section .data, size 0x28 lma 0x8000f30
Start address 0x8000b4c, load size 3928
Transfer rate: 6 KB/sec, 1964 bytes/write.

(gdb) b fonction1       ← il s'agit de la fonction appelée au début du programme,
                          le but ici est seulement de s'arrêter dès que toutes
                          les configurations nécessaires auront été faites
Breakpoint 1 at 0x80002ae: file firmware.c, line 71.

(gdb) c
Continuing.
Note: automatically using hardware breakpoints for read-only addresses.
Breakpoint 1, fonction1 (v=40960) at firmware.c:71
71      fonction1();

(gdb) x /x 0x40020C00
0x40020c00:     0x55000000      ← Pourquoi cette valeur ?

(gdb) set *0x40020C14=0x1000    ← La led verte s'allume, pourquoi ?

(gdb) set *0x40020C14=0x2000    ← La led orange s'allume, pourquoi ?

(gdb) set *0x40020C14=0x4000    ← La led rouge s'allume, pourquoi ?

(gdb) set *0x40020C14=0x8000    ← La led bleue s'allume, pourquoi ?

(gdb) set *0x40020C14=0xF000    ← Toutes les leds s'allument, pourquoi ?
```

Rappel concernant gdb :
x /x *: affiche le contenu de la mémoire à une adresse donnée, en hexadécimal*
set *: affecte une valeur au contenu de la mémoire à une adresse donnée*

Exercice 3.

Nous commençons maintenant l'écriture du code C qui permettra de réaliser l'équivalent de ci-dessus (allumages leds).

Q1. Rappeler **ce que signifie** exactement la valeur

```
(* (volatile uint32_t *)0x40020C14 )
```

pour la macro `GPIOD_ODR`. En particulier, s'agit-il de l'adresse du registre ou de son contenu ?

Q2. Quel est (en hexadécimal, en décimal) **l'intervalle de valeurs** à utiliser pour le choix d'allumage des leds (zéro, une, ou plusieurs leds) ?
Quels bits du registre de la question 1 devront recevoir l'une des valeurs dans cet intervalle ?

Q3. Rappeler le **principe** (les affectations à faire) selon lequel une valeur V dans cet intervalle pourra être placée au bon endroit dans le registre, sans modifier les autres bits.
Ecrire le **code C** (les 2 instructions) correspondant.

Q4. Ecrire la **fonction** *write_leds_int* qui prend en paramètre un entier non signé sur 16 bits, dans les faits compris entre 0 et 15 (modélisant l'allumage choisi), et place cette valeur à l'emplacement voulu dans ODR.